

CRDTs and Databases: A Hands-On Tutorial

Part 3 – CRDVs Hands-On

Paulo Sérgio Almeida, Nuno Faria, José Pereira

psa@di.uminho.pt, nuno.f.faria@inesctec.pt, jop@di.uminho.pt



INESCTEC



Universidade do Minho
uminho.pt

VLDB 2026

CRDVs in PostgreSQL

Timestamp Encoding

- Array

key	...	lts
k1	...	[2, 1]
k2	...	[2, 3]

- JSON

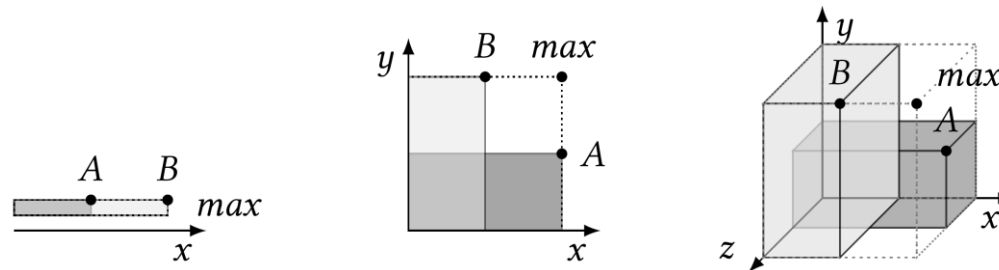
key	...	lts
k1	...	{"1":2, "2":1}
k2	...	{"1":2, "2":3}

- Relational

key	...	key	site	t
k1	...	k1	1	2
k2	...	k1	2	1
		k2	1	2
		k2	2	3

- Geometric

key	...	lts
k1	...	(0, 0), (2, 1)
k2	...	(0, 0), (2, 3)



CRDVs in PostgreSQL



INESCTEC



Universidade do Minho
uminho.pt

Timestamp Encoding

- Custom type for vector clocks.

```
CREATE DOMAIN vclock AS bigint[];
```

- Custom functions are implemented in both SQL and C.

```
CREATE FUNCTION vclock_lte(v1 vclock, v2 vclock)
RETURNS bool LANGUAGE SQL AS $$
    SELECT bool_and(
        coalesce(u1, 0) ≤ coalesce(u2, 0)
    )
    FROM (
        SELECT unnest(v1) AS u1, unnest(v2) AS u2
    ) t;
$$;
```

```
CREATE FUNCTION
    vclock_lte(v1 vclock, v2 vclock)
RETURNS Boolean AS
    '$libdir/clocks', 'vclock_lte'
LANGUAGE C STRICT;
```

CRDVs in PostgreSQL



INESCTEC



Universidade do Minho
uminho.pt

Schema

- Generic History (**Shared**) and Materialized Present (**Local**) tables to represent a variety of CRDTs.

id	key	type	data	site	lts	pts	op
----	-----	------	------	------	-----	-----	----

- View to compute causal Present data (**Data**):
 - Sync mode
 - Async mode

```
SELECT *  
FROM Local;
```

```
SELECT ... FROM (  
  WITH potential_max AS (...) -- most recent rows in at least one site  
  SELECT t1.*,  
         NOT vclock_lte(t1.lts, t2.lts) OR t1.lts = t2.lts lte  
  FROM potential_max t1  
  JOIN LocalAndShared t2 ON t1.id = t2.id AND t1.key = t2.key  
) t  
GROUP BY ... HAVING bool_and(lte) = true;
```

CRDVs in PostgreSQL



INESCTEC



Universidade do Minho
uminho.pt

Writes

- Writes are sent to the causal Present view, which in turn redirects them to History using rules.

```
CREATE RULE Data_insert_rule AS
ON INSERT TO Data
DO INSTEAD
INSERT INTO Shared VALUES(new.id, new.key, ...);
```

- Sync mode:** trigger automatically merges rows into the materialized Present and cleans them from History.
- Async mode:** the trigger is disabled.

```
BEGIN
PERFORM merge(new.id, new.key, ...);
DELETE FROM Shared ...
RETURN new;
END;
```

Merge

- Discards obsolete materialized Present data and inserts the new row. Runs at each write (sync) or periodically with a pg_background worker (async).

```
CREATE OR REPLACE FUNCTION merge(id_ varchar, key_ varchar, ...)
RETURNS void AS $$
BEGIN
    -- lock by key to avoid conflicts
    PERFORM _access_elem(id_ || '.' || key_);

    -- determine if already obsolete
    IF _is_operation_obsolete(id_, key_, lts_) THEN RETURN; END IF;

    -- remove obsolete entries
    PERFORM _delete_past_ops(id_, key_, lts_);

    INSERT INTO Local VALUES (id_, key_, ...);
...

```

Replication

- Inserts to the History table are propagated to remote sites using Publications and Subscriptions (uses asynchronous logical replication).

- Publication:

```
CREATE PUBLICATION Shared_Pub
FOR TABLE Shared
WHERE (site = %s) -- only send the local rows
WITH (publish = 'insert');
```

- Subscriptions (one per remote site):

```
CREATE SUBSCRIPTION sub_%s_%s
CONNECTION 'host=%s port=%s dbname=%s user=%s password=%s'
PUBLICATION Shared_Pub
WITH (create_slot = false);
```

Implementing common CRDTs

- Registers: $r1 = abc$

id	key	type	data	...
r1		r	abc	...

- Maps: $m1 = \{k1:xyz, k2:123\}$

id	key	type	data	...
m1	k1	m	xyz	...
m1	k2	m	123	...

- Sets: $s1 = \{a, b\}$

id	key	Type	data	...
s1	a	s		...
s1	b	s		...

- Lists: $l1 = [a, b, c]$ ('P1' < '[1' < 'g1')

id	key	type	data	...
l1	P1	l	a	...
l1	g1	l	c	...
l1	[1	l	b	...

Index generation
based on the LSEQ
algorithm.

Brice Nédelec, Pascal Molli, Achour Mostefaoui, and Emmanuel Desmontils. 2013. LSEQ: an adaptive structure for sequences in distributed collaborative editing. In Proceedings of the 2013 ACM symposium on Document engineering.

- Counters: $c1 = 7+5 = 12$

id	key	type	data	...
c1	1	c	7	...
c1	2	c	5	...

One entry per site.

Implementing common CRDTs

- Resolution views and utility functions are implemented for each type.

```
-- add wins + mvr for concurrent adds for maps
```

```
CREATE VIEW MapAwMvr AS
```

```
  SELECT id as id, (key, array_agg(data ORDER BY site))::mEntryMvr data  
  FROM Data
```

```
  WHERE type = 'm' AND op = 'a'  
  GROUP BY id, key;
```

```
-- add an entry to a map
```

```
CREATE FUNCTION mapAdd(id_ varchar, key_ varchar, value_ varchar) RETURNS void AS $$  
BEGIN
```

```
  INSERT INTO Data (id, key, type, data, site, lts, pts, op)
```

```
  SELECT id_, key_, 'm', value_, siteId(), (t).lts, (t).pts, 'a'
```

```
  FROM nextTimestamp(id_) AS t;
```

```
END
```

```
$$ LANGUAGE PLPGSQL;
```

Demo

- Implemented in the browser with embedded Postgres databases (PGLite).
- Logical replication is not available, so replication is done manually.
- Available at: nuno-faria.github.io/crdv/demo



The screenshot displays the CRDV demo interface, which manages two database sites. At the top, a timeline shows the sequence of operations: Site 1 updates at 12:12:20, Site 2 updates at 12:12:25, Site 1 updates at 12:12:27, and Site 2 updates at 12:12:28. A 'Replicate and Merge' button is positioned between the site panels. Each site panel includes a query input area, a 'Run' button, and buttons for 'Query Present', 'Query History', and 'SQL Log'. The results for the query 'SELECT * FROM mapAwLww' on Site 1 and 'SELECT * FROM mapAwMvr' on Site 2 are shown below the input areas.

Site 1 (idb://pg-site-1: ready)

```
SELECT * FROM mapAwLww
```

[Ctrl/⌘ + Enter] to run

Run Query Present Query History SQL Log

id	data
m	(k1, v1)
m	(k2, v2)
m	(k3, v30)

(3 rows)

Site 2 (idb://pg-site-2: ready)

```
SELECT * FROM mapAwMvr
```

[Ctrl/⌘ + Enter] to run

Run Query Present Query History SQL Log

id	data
m	(k1, {v1})
m	(k2, {v2})
m	(k3, "{v3, v30}")

(3 rows)



Demo: Shopping cart example

- Populate data:

```
SELECT mapAdd('cart-1', 'Apples', 'qnt-1');
SELECT mapAdd('cart-1', 'Bananas', 'qnt-2');
SELECT mapAdd('cart-1', 'Cherries', 'qnt-3');
SELECT counterInc('qnt-1', 2);
SELECT counterInc('qnt-2', 2);
SELECT counterInc('qnt-3', 3);
```

```
SELECT mapAdd('cart-2', 'Apples', 'qnt-4');
SELECT mapAdd('cart-2', 'Bread', 'qnt-5');
SELECT counterInc('qnt-4', 1);
SELECT counterInc('qnt-5', 1);
```

```
store = {
  "cart-1" = {
    "Apples": Counter(2),
    "Bananas": Counter(2),
    "Bread": Counter(3),
  },

  "cart-2" = {
    "Apples": Counter(1),
    "Bread": Counter(1),
  }
}
```

Demo: Shopping cart example

- Resolution views reuse existing rules:

```
CREATE VIEW CartAw AS
  SELECT m.id, (m.data).key as product, c.data as quantity
  FROM MapAwLww m
  JOIN Counter c ON (m.data).value = c.id
  WHERE m.id LIKE 'cart-%';
```

```
CREATE VIEW CartRw AS
  SELECT m.id, (m.data).key as product, c.data as quantity
  FROM MapRwLww m
  JOIN Counter c ON (m.data).value = c.id
  WHERE m.id LIKE 'cart-%';
```

Demo: Shopping cart example

- Single write:

```
SELECT counterDec('qnt-1', 1);
```

- Concurrent write (additive):

```
-- site 1  
SELECT counterInc('qnt-3', 1);  
-- site 2  
SELECT counterInc('qnt-3', 1);
```

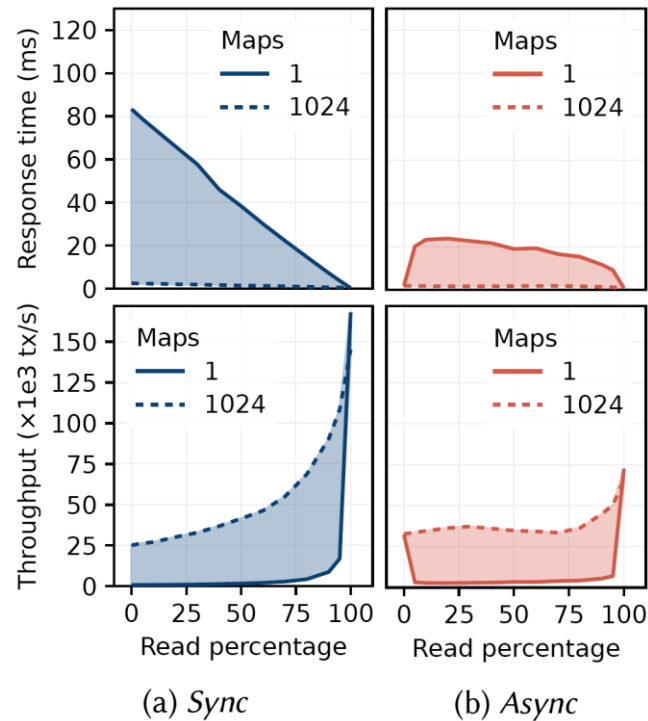
- Concurrent add and remove (different structures):

```
-- site 1  
SELECT mapRmv('cart-2', 'Apples');  
-- site 2  
SELECT counterInc('qnt-4', 1);
```

- Concurrent add and remove that ensures add-wins:

```
-- to revert the previous writes  
SELECT mapAdd('cart-2', 'Apples', 'qnt-4');  
SELECT counterDec('qnt-4', 1);  
  
-- site 1  
SELECT mapRmv('cart-2', 'Apples');  
-- site 2  
BEGIN;  
SELECT counterInc('qnt-4', 1);  
SELECT mapAdd('cart-2', 'Apples', 'qnt-4');  
COMMIT;
```

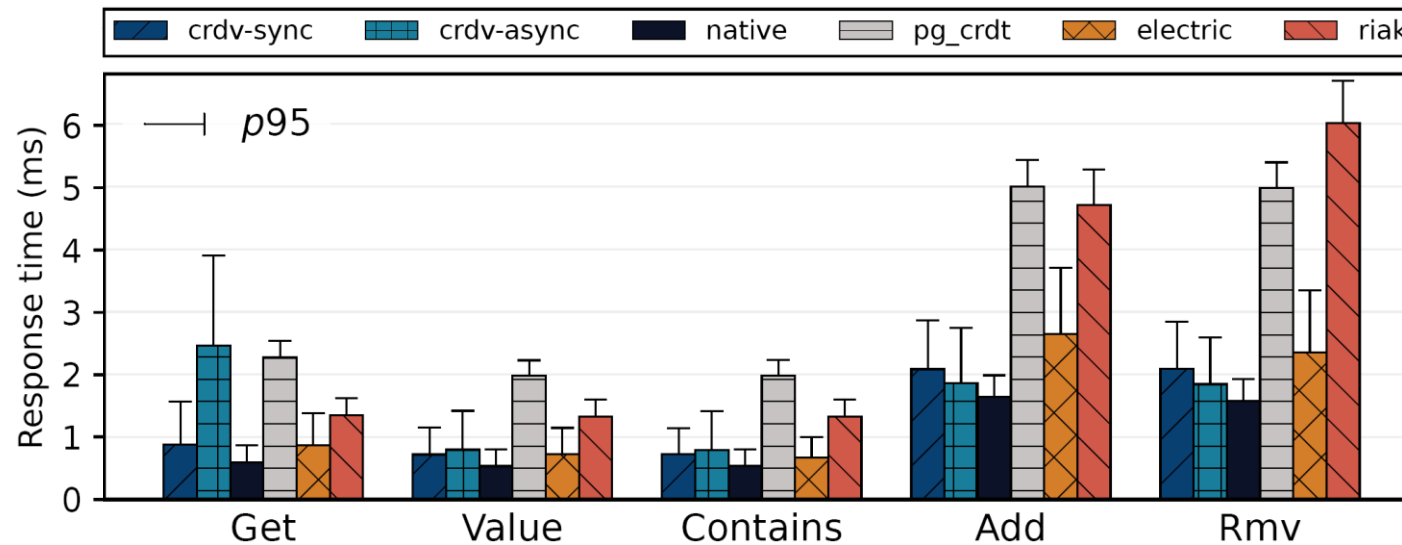
Sync vs Async mode



- Sync mode is better for a higher percentage of reads.
- Async mode is better for a higher percentage of writes, especially with hotspots.

Response time and throughput based on the percentage of reads/writes and number of maps (64 clients).

CRDT Operations

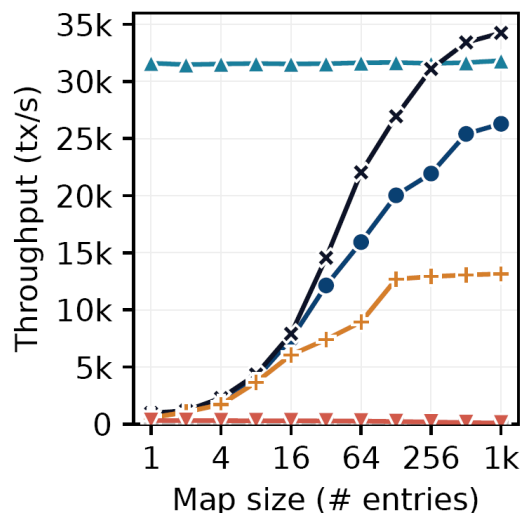
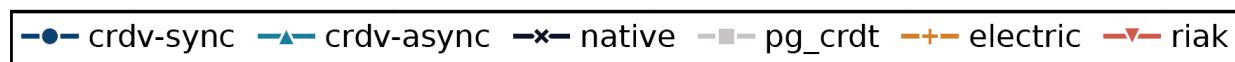


(e) Map

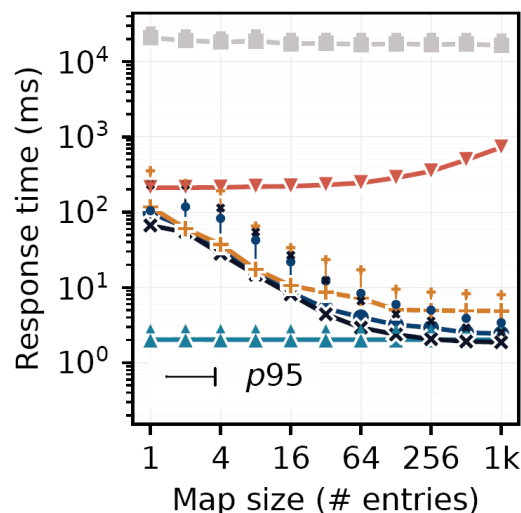
Response time for different operations over a single map (1 client).

- CRDV-sync read performance is close to native Postgres, beating alternative solutions.
- CRDV-async is slower when materializing the entire map (Get) but faster when retrieving one key (Value, Contains)
- CRDV is faster than the alternatives when writing.
 - Especially faster than Pg_crdt and Riak, since they rewrite the entire map.

Concurrency



(a) Throughput



(b) Response time

Throughput and response time for writes over a single map, based on the number of entries (64 clients).

- CRDV-async avoids contention by skipping the merge.
- Pg_crdt and Riak rewrite the entire structure, so even writes over different keys conflict with each other.

Performance Evaluation

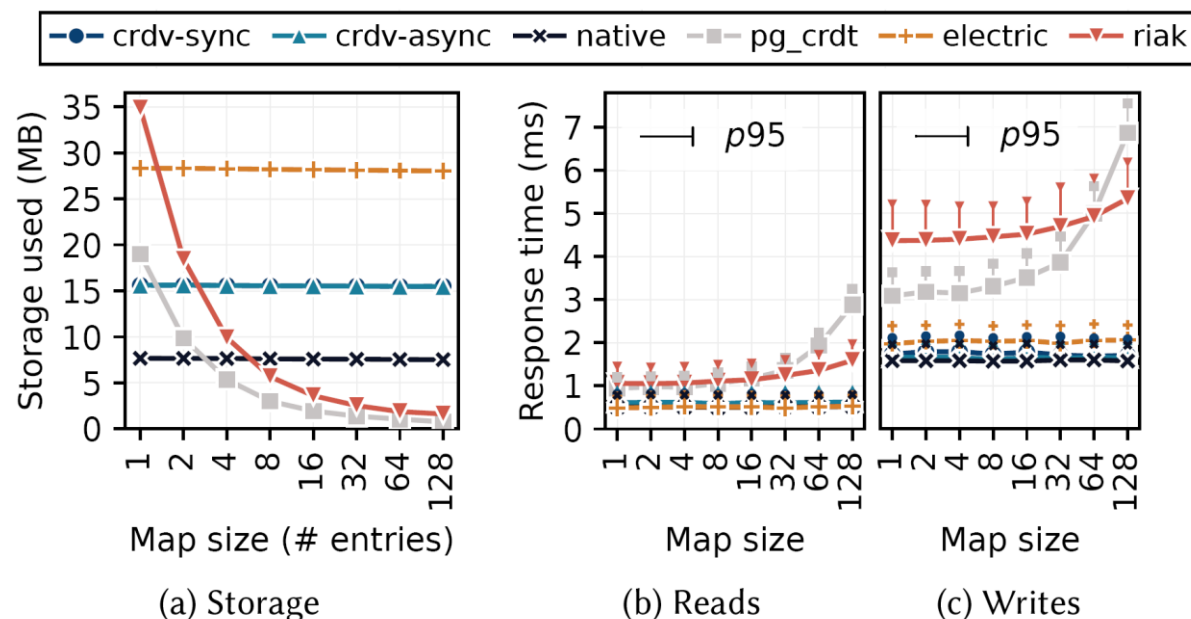


INESCTEC



Universidade do Minho
uminho.pt

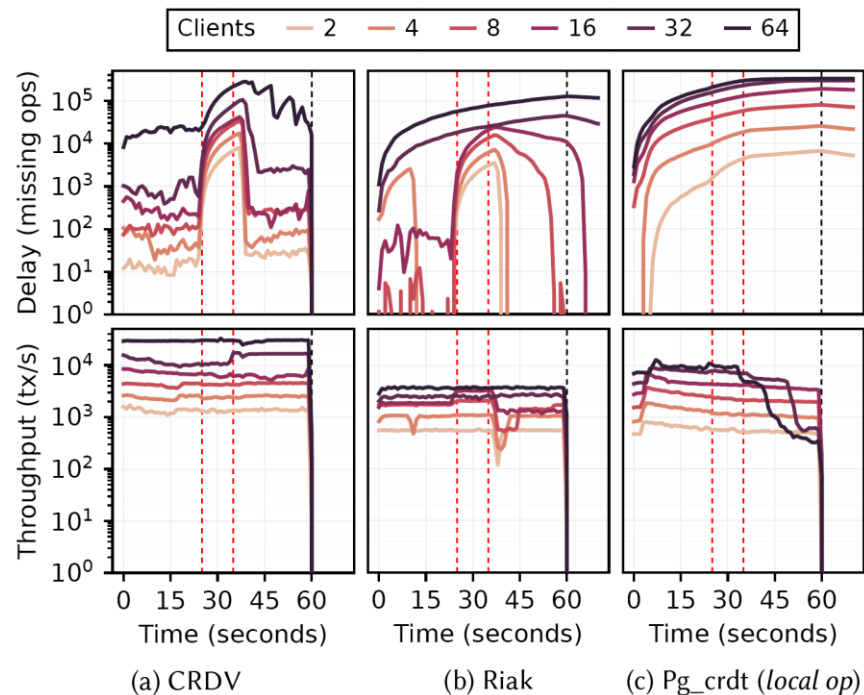
Storage



Storage and latency with 100k kv pairs, based on the number of maps (e.g., $x = 1 \rightarrow 100k$ maps of size 1, $x = 2 \rightarrow 50k$ maps of size 2).

- CRDV's storage size depends on the number of rows alone.
- Pg_crdt and Riak store one timestamp per structure, so they require less space with less maps.
- Conversely, read and write latencies increases for Pg_crdt/Riak since the entire structure is decoded/encoded each time.

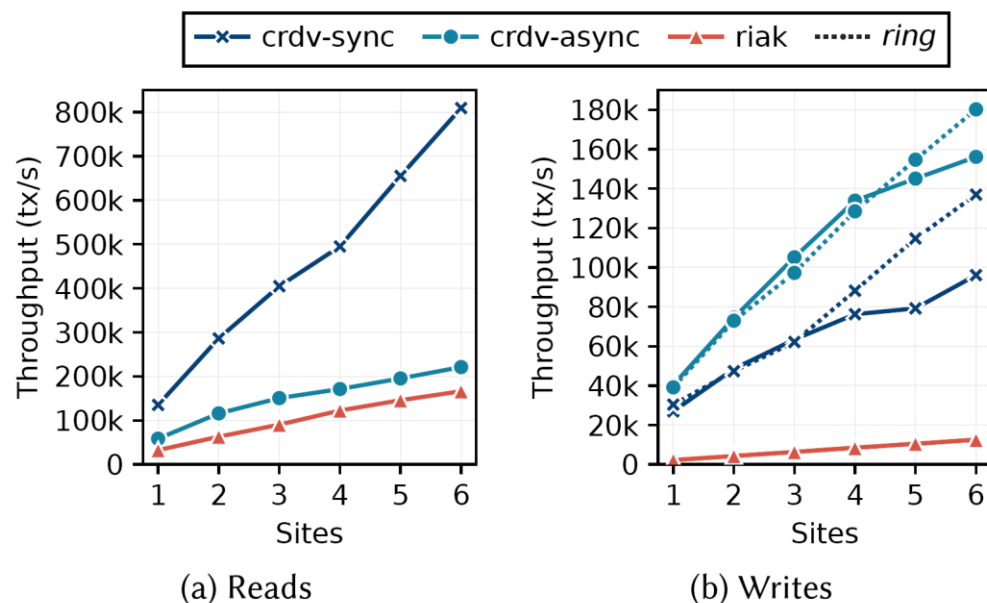
Freshness



Total replication delay and throughput over time in three sites, based on the number of clients (network is temporarily partitioned between t=25s and t=35s).

- Even at a higher throughput, CRDV manages to maintain the replication delay stable, mainly due to merging remote rows in batch (i.e., one disk sync per multiple rows).
- Pg_crdt (with just 2 clients) and Riak (with 32 or more clients) are not able to keep up with the replication.

Scalability



Total read and write throughput based on the number of sites.

- All systems scale reads linearly with the additional sites, although at different baselines.
- Both CRDV modes scale writes until 4 sites. After that, the replication overhead of a fully connected architecture becomes the main bottleneck.
- Changing to a ring architecture improves the scalability.

CRDTs and Databases: A Hands-On Tutorial

Part 3 – CRDVs Hands-On

Paulo Sérgio Almeida, Nuno Faria, José Pereira

psa@di.uminho.pt, nuno.f.faria@inesctec.pt, jop@di.uminho.pt



INESCTEC



Universidade do Minho
uminho.pt

VLDB 2026

This work is financed by National Funds through the Portuguese funding agency, FCT - Fundação para a Ciência e a Tecnologia; Paulo Almeida under the support UID/50014/2025 (DOI:10.54499/UID/50014/2025); Nuno Faria and José Pereira by CMU-Portugal project ADAPQO (DOI:10.54499/2024.12585.CMU).